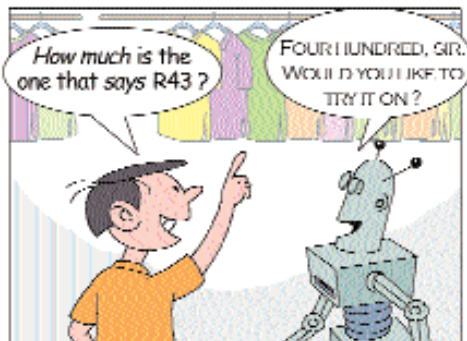
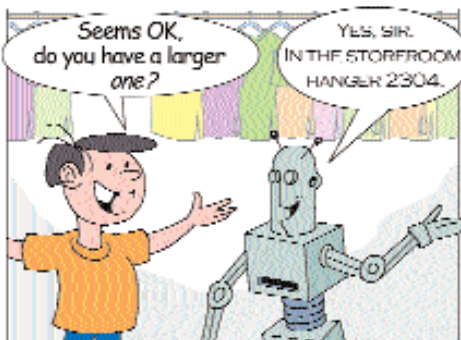




'Price', not 'quantity'; and 'R43' was not said by someone



It realised 'size of T-shirt', not of something else

that can be done when parsing programs. Even the composition of a very simple sentence can be seen as a complex combination of noun phrases, verb phrases and so on. Determining these is not an easy task; issues such as the order of the phrases come into play. For example, a verb phrase such as 'killed the mouse' always follows, (and never precedes) a noun phrase, such as 'Garfield', hence—'Garfield killed the mouse.'

Once this is done, organising the components into semantic relationships would be the natural step, because from established knowledge, we know the roles of those components. This semantic representation can be in the First Order Predicate Calculus (FOPC). The FOPC allows one to make assertions in a formal way. If you had to say, "There are people who cross this bridge, but they do it only between 6:00 pm and 8:00 pm", this might be expressed in the FOPC as follows: $(P: (C)) \text{ AND } (C: (C = B))$, where means 'there exist', x means 'for all x ', P is 'people', C is 'crossing the bridge' and B is 'crossings between 6 and 8'. You can reason using the FOPC. Suppose you had

two assertions—'all cats are alive', and 'Garfield is a cat'. When you express these in the FOPC, you would have a semantic representation of those two facts, and you could automatically deduce that Garfield is alive.

In order to bridge the gap between the syntax and the semantics, we have to reason with the parsed sentences. The Lambda Calculus is one way of doing this. It allows one to express an idea with the actual details removed; say you want to talk about 'redness' without referring to a red object like an apple. You could say that " $\lambda x.(\text{expression_containing_}x)$ " means 'the role of the x in that expression'. With x standing for 'apple' and 'expression_containing_x' being 'a red apple', that entire Lambda expression would mean "redness." An expression in the Lambda Calculus can take one or more arguments, and results in a concrete expression that is the result of the idea applied to that concrete argument—so when that entire expression is applied to 'tables', it would result in 'red tables'.

But how do we reason with, for example, 'Jon likes Garfield'? We need a

The Loebner Prize

In the 1950's, Alan Turing made the famous statement that if a computer's responses to a human questioner left the human in doubt as to whether he were talking to a computer or a human, then the computer could be said to be thinking. That statement is extremely debatable, and debated too. Anyway, an award called the Loebner Prize seems to be built on Turing's state-

ment. It judges whether a computer's speech can fool a human listener into thinking it's a person speaking. The Grand Loebner Prize receiver is the machine that fools everyone that talks to it, and the annual prize is for the programmer of the most human machine each year. Visit <http://www.loebner.net/Prize/loebner-prize.html> to know more.



Microsoft Wheel Mouse Optical



Reliable & Affordable

- Intelli Eye Optical Technology
- Ambidextrous Ergonomic Design
- Complete Reliability at a great value
- 3 Year Replacement Warranty

Microsoft